Skill Misfit after Injection: Revising and Withdrawing Retrieved Skills for LLM Agents

Anonymous ACL submission

Abstract

LLM agents increasingly load *skills*: short procedures retrieved for a task and injected into the prompt. Injected once and never revisited, a skill that fits the goal can mislead the agent later. On ALFWorld's text-based games, restoring the same states shows that the skills alone cause failures: from states of games a reasoning agent wins only without skills, continuing with the skills fails all 18 times, whereas withdrawing them wins 11. We propose TRIM, a training-free method that scores how much the skills raise the likelihood of the agent's drafted response, reasoning included: when they drive it, an Editor revises them; when they drive it again, they are withdrawn for the rest of the episode. With Qwen3-8B in thinking mode, TRIM wins 84, 80 and 75 of 134 unseen test games in three runs, on average 19.7 more than the retrieved skills (14.7 points, 95% CI [8.7, 20.9]). It also leads with Qwen3-14B and on SRA-Bench ToolQA and beats one edit at injection and both stages at random steps. Withdrawing only after a failed revision keeps most of the skills' help: TRIM loses 4 of their 60 test wins (run 1), against 24 without them.

1 Introduction

Agent skills package procedures, tool use and applicability conditions into short texts that an agent can reuse across tasks (Anthropic, 2025; Li et al., 2026b; Xia et al., 2026). A skill is written once, before the tasks it will serve are seen, and one skill serves many tasks; at run time the agent retrieves several skills from the task description and injects them into its prompt. Whether the injected skills fit a given episode is therefore a separate question.

Skills do not always help: a skill that misfits the episode can lead the agent into a wrong action, add cost without changing the outcome, or leave a failure unsolved (Dong et al., 2026; Han et al., 2026; Wang et al., 2026c; Li et al., 2026a). Part of this misfit is visible at injection, when a retrieved item does not fit the task. Part of it appears only as the episode unfolds: observations reveal what the task description does not, such as what the environment holds or which attempts have failed, so a skill that suited the task can stop suiting the current state while its text stays in the prompt and keeps steering later decisions.

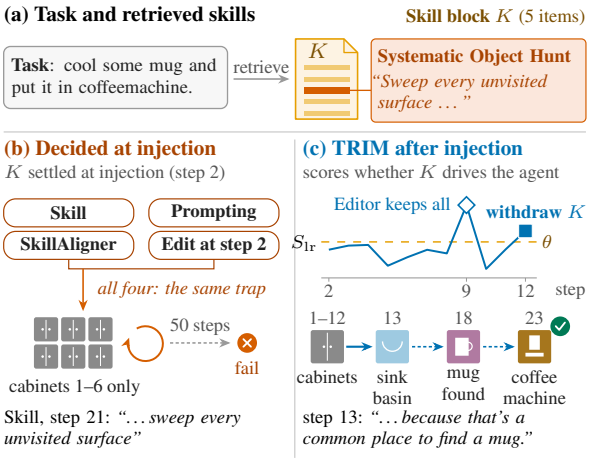


Figure 1: One test game (App. G). (a) The block retrieved from the goal holds a search routine. (b) Arms that settle the skills at injection stay in cabinets 1–6 for 50 steps. (c) TRIM has Skill's prompts up to step 12; the Trigger fires at step 9 (the Editor keeps all items) and at step 12 (block withdrawn), after which the agent leaves the cabinets and finds the mug. The agent without skills also finds it (32 steps). Icons in (c) skip steps.

Existing remedies address the first kind: they decide when and which skills enter the context (Chen et al., 2026; Li et al., 2026c), retrieve skills for each plan step (Wang et al., 2026a), rewrite them into a task guide (Li et al., 2026a), or refine the library between episodes (Zhang et al., 2026b; Ouyang et al., 2026; Liu et al., 2026b). Each settles the skill text at injection or between episodes; once a skill is in the context, none of them revisits it against what the episode reveals.

Figure 1 shows the second kind: the block retrieved for cooling a mug holds a search routine;

with it, the agent and every method that settles the skills at injection search cabinets 1–6 for all 50 steps, the Skill agent still citing it at step 21 ("sweep every unvisited surface"), whereas the agent without skills finds the mug. An edit at step 2 keeps the search item, and the SkillAligner guide also ends there.

TRIM therefore revisits the skills after injection, with a *Trigger* that finds the steps the skills drive and an *Editor* that revises them there (Figure 2b). The Trigger scores the drafted response, reasoning included, with and without the skill block and takes the log-likelihood ratio, S_{lr}, as the skills' influence on the step; it fires when S_{lr} crosses a threshold set as an intervention budget, without outcome labels. A firing does not mean that the skills are wrong, only that they drive this step, making it a candidate for misfit. At the first firing the Editor checks the items against the task and the observations and keeps, withholds or replaces them; if the reviewed block fires the Trigger again, the pull comes from the block itself, and TRIM withdraws it for the rest of the episode, not earlier, since removing skills also removes their help.

We evaluate on the 134 unseen test games (household tasks) of ALFWorld (Shridhar et al., 2021) and on 300 questions of SRA-Bench ToolQA (Su et al., 2026; Zhuang et al., 2023), with Qwen3-8B and Qwen3-14B in thinking mode (Yang et al., 2025). TRIM wins 84, 80 and 75 games in three runs, on average 19.7 more than the retrieved skills (14.7 points, 95% CI $[8.7, 20.9]$; 28:4 games in its first run, $p < 0.001$); it also leads with Qwen3-14B and on ToolQA, and beats one edit at step 2 and both stages at random steps. Our contributions are:

- A study of skill misfit with a reasoning agent: from the same states, the retrieved skills win none of 18 continuations, whereas withdrawing them wins 11 (§3).
- TRIM, a training-free method in which the drafted response's likelihood with and without the skills decides when to revise and then withdraw them; to our knowledge, no prior method uses a model-internal signal to edit or withdraw an injected skill mid-episode (§4).
- Experiments on ALFWorld and ToolQA with two backbones against five baselines and two timing controls, ablations of each component, and an analysis showing that withdrawal must be selective (§5–6).

2 Related Work

Agent skills. Agents keep experience as reusable skills (Wang et al., 2024; Zhao et al., 2024; Wang et al., 2025; Xia et al., 2026); packaged as agent skills in the prompt (Anthropic, 2025), curated skills raise average pass rates (Li et al., 2026b). Later work refines skill libraries (Zhang et al., 2026b; Ouyang et al., 2026; Liu et al., 2026b), retrieves skills for each plan step (Wang et al., 2026a), learns whether to invoke one at each step (Chen et al., 2026) or which to inject (Li et al., 2026c), and adapts them into a task guide (Li et al., 2026a); most settle the skill text at injection or between episodes, whereas TRIM revises it during the episode.

Skill misfit and adaptation. Skills can also cause failures an agent would not make without them (Dong et al., 2026): gains come with regressions (Tank and Nama, 2026), not every skill helps (Wang et al., 2026c), and gains shrink under realistic retrieval (Liu et al., 2026a); a retrieved skill can help in one state and mislead in another (Tu et al., 2026). Our pairing (Table 1; §3) adds a rerun-noise baseline and continuations from the same states with and without the skills. Other methods adapt context during an episode: they re-retrieve exemplars by the agent's current thought, select guidelines for the identified context, or revise plans from environment feedback (App. A); none of them checks whether the injected skills are driving the agent. Self-correction refines an output within one attempt (Madaan et al., 2023) or reflects after failed attempts (Shinn et al., 2023), and often fails without external feedback (Huang et al., 2024); TRIM edits the skill block, checking it against the task and the observations.

Signals for intervening. TRIM's Trigger decides from the model's own likelihoods when to intervene. Adaptive retrieval decides when to consult external knowledge from uncertainty and internal states (Jiang et al., 2023; Su et al., 2024), and concurrent work reads model internals to select skills (Lin et al., 2026) or to localize turns for offline skill revision (Zhang et al., 2026a). TRIM instead revises and, if the skills keep driving the agent, withdraws them within one episode; its Trigger compares the likelihood of the drafted response with and without the skills, a pointwise CXMI (Fernandes et al., 2021, 2023) that PECoRe thresholds (Sarti et al., 2024) (other uses: App. A).

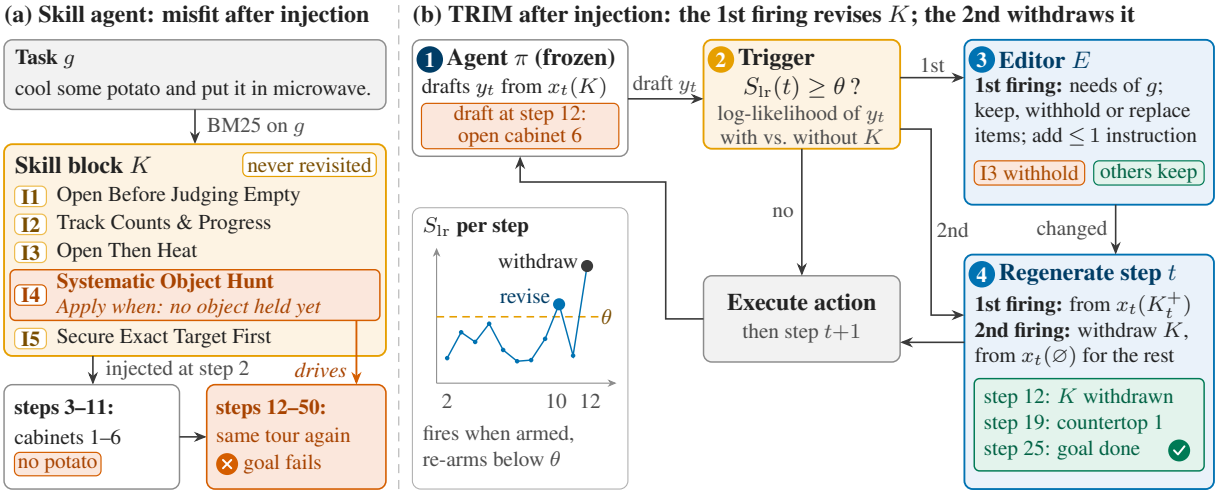


Figure 2: TRIM on one ALFWorld game (Qwen3-8B, thinking; App. G). (a) The block retrieved from the goal is injected at step 2 and never revisited: while no object is held, the search item I4 keeps applying, and the Skill agent tours the cabinets and the fridge until step 50. (b) After injection, the armed Trigger fires when the block drives the drafted response ($S_{\mathrm{lr}} \geq \theta$): the first firing sends the block to the Editor (step 10: the heating item I3 is withheld), the second withdraws it (step 12); each change regenerates the step, and without the block the agent wins at step 25.

3 Motivating Analysis

3.1 Problem Setting

We consider an LLM agent π that solves a task, described by a goal g, by interacting with an environment. At each step t the agent receives an observation o_t and the set of admissible actions $\mathcal{A}_t$, and selects an action a_t; the episode succeeds ($R = 1$) if the goal is reached within T steps and fails ($R = 0$) otherwise. We write $\tau_t = (o_0, a_1, o_1, \ldots, o_t)$ for the history up to step t.

Skills are retrieved once, from the goal alone. Given a skill library $\mathcal{L}$ and a relevance score r, the retriever returns the skill block

$$K = \underset{s \in \mathcal{L}}{\text{top-}k}\ r(g, s) = \{\kappa_1, \ldots, \kappa_k\}, \qquad (1)$$

whose items κ_j are short procedures, some stating when they apply. The block enters the prompt at step t_0, and K_t denotes the block in effect at step t, so that $K_t = K$ unless it is changed during the episode. The agent then generates a reasoning trace z_t followed by an action,

$$y_t = (z_t, a_t) \sim \pi\big(\cdot \mid x_t(K_t)\big), \qquad (2)$$

where $x_t(B)$ is the prompt built from g, the recent history, o_t and $\mathcal{A}_t$ with block B, and $x_t(\varnothing)$ is the same prompt without skills. In our experiments r is BM25, $k = 5$, $t_0 = 2$ and $T = 50$ (App. C.1).

Misfit after injection. Since K is fixed from the goal alone, whether it helps depends on the states the episode reaches. Let $V(\tau_t, B) = \Pr[R = 1 \mid \tau_t, B]$ be the probability of success when the episode continues from τ_t with block B in the prompt. We say that the block misfits at τ_t if some modification B' of K_t, possibly the empty block, attains $V(\tau_t, B') > V(\tau_t, K_t)$; a block that fits at t_0 can therefore misfit later (§3.2). As V cannot be observed during an episode, the goal is to decide from signals available to the agent itself when and how to change the block, setting $K_{t+1} \in \{K_t,\ E(K_t),\ \varnothing\}$ with the Editor E (§4.2), so that a misfitting block is changed, a helpful one is kept, and interventions remain rare.

3.2 Empirical Findings

We compare the agent without the skill block (NoSkill) and with the block as retrieved (Skill), using Qwen3 in thinking mode on ALFWorld (§5.1); counts come from the first test run unless the development games (the 140 `valid_seen` games of ALFWorld, used to design the method) are named.

Outcome changes. The block changes far more outcomes than a second run of the same arm, and its losses nearly cancel its gains (Table 1a): with Qwen3-8B it loses 21 of NoSkill's 57 test wins while winning 24 other games. By task type, the block gains mostly on Clean (12 games against 2 lost) and loses on Cool (8 lost, 5 gained) and PickTwo (4 lost, none gained).

Same-state continuations. To hold the history fixed, we restored 12 states of development

(a) Outcome changes between two arms or two runs

Reference → compared	Changed	Lost	Gained	Net
Test: NoSkill → Skill	45	21	24	+3
Test (14B): NoSkill → Skill	32	15	17	+2
Dev.: NoSkill → Skill	35	14	21	+7
Dev.: NoSkill → rerun	19	7	12	+5
Dev.: Skill → rerun	13	7	6	−1

(b) Same-state continuations: wins

Restored states	Keep	Edit	No block
Harmed (6 states)	0/18	8/18	11/18
Helped (3 states)	7/9	6/9	6/9

Table 1: Misfit evidence on ALFWorld (Qwen3-8B unless marked; thinking mode). (a) Games whose outcome differs (Changed), as wins of the reference that the compared arm loses (Lost) or games it wins that the reference loses (Gained); Net: change in wins. Test: run 1 on the 134 test games; Dev.: development games. (b) Wins of three continuations from each restored development state with the block kept, edited or removed (App. B.1).

episodes with the block, chosen by the games' outcomes (App. B.1), and continued each three times with the block unchanged, edited by the Editor of §4.2 without the reasoning excerpt, and removed. These continuations estimate $V(\tau_t, K)$, $V(\tau_t, E(K))$ and $V(\tau_t, \varnothing)$ at the same τ_t; as each is sampled afresh, the choice of states fixes where they start, not how they end. From the harmed states, the unchanged block wins none of 18 continuations, the edited block 8 and the agent without it 11 (Table 1b): the same state fails with the skills and succeeds without them. Where the skills help, removing them costs one win in nine.

Failure patterns. Reading the 15 development games that the skills harm, we find two patterns (App. B.2). In nine, the agent never finds the target: 91% and 98% of its moves in the two Skill runs go to closed containers, against 59% and 63% without skills, and in six its reasoning cites a search item ("Systematic Object Hunt") in 37 to 48 of 49 steps. In four, BM25 attached a procedure written for another task type through a shared word, such as a cooling item in a heating task whose destination is the fridge; across the development heating games whose block holds that item, Skill wins none of 8 in either run and NoSkill 2 and 4. The other two failures are unrelated to the skills.

Accumulation within an episode. A search item does its harm by repetition: in the game of Figure 2, Skill circles through six cabinets and the fridge until step 50 and still cites "Systematic Object Hunt" at step 21, whereas NoSkill checks the countertop at step 3 and wins in 9 steps. A procedure for another task acts once its state arrives: in a development game that asks to heat a tomato and put it in the fridge, the agent cools the tomato right after heating it at step 26 because "the 'Enforce Cooling Before Placement' takes precedence", heats it again at step 45 and cools it again at step 47. Neither failure is visible in the task description, and both build up over steps that the retrieved text keeps steering.

Implications for the method. Two properties of misfit shape TRIM. Misfit appears only with observations that the task description does not contain, so the block must be revisited after injection, at the steps it steers. The two failure patterns also need different changes: a misattached item can be withheld or replaced while the items that help stay, whereas a search routine spans several items and steers the agent over many steps, which an item-level edit can leave in place. Because removing the block recovers the most harmed states but also costs wins where the skills help, TRIM edits the block first and withdraws it only if the reviewed block again drives the agent.

4 Method

TRIM leaves the agent unchanged and controls only which skill block it sees: a *Trigger* decides when to change the injected block by scoring how strongly it drives the agent's next response (§4.1), and a *two-stage intervention* decides how, with an *Editor* revising the block at the first firing and withdrawal at the second (§4.2; Figure 2b, Algorithm 1).

4.1 Trigger

A skill can mislead the agent only at the steps it drives: if the drafted response is about as likely without the block, changing the block changes little, whereas if it is much more likely with it, the reasoning and the action follow the retrieved text. The Trigger therefore asks how much less likely the agent would be to produce the draft without the skills, not, as uncertainty signals in adaptive retrieval do (Jiang et al., 2023; Su et al., 2024), whether it lacks information. For the drafted response y_t it compares each token's log-probability with and without the block,

$$c_{t,i} = \log \frac{\pi\big(y_{t,i} \mid x_t(K_t),\, y_{t,<i}\big)}{\pi\big(y_{t,i} \mid x_t(\varnothing),\, y_{t,<i}\big)}, \qquad (3)$$

Algorithm 1 TRIM in one episode. The first firing sends the block to the Editor; a second firing withdraws it for the rest of the episode.

Require: goal g; block K (BM25 on g); agent π; Editor E; θ

$armed \leftarrow$ **true**; $n \leftarrow 0$ ▷ firings so far
for $t = 1, \dots, 50$ **do**
  $y_t \leftarrow \pi\big(x_t(K)\big)$ ▷ reasoning, then action
  if $t \geq t_0$ **and** $K \neq \varnothing$ **then**
    $s \leftarrow S_{\mathrm{lr}}(t)$ ▷ Eq. (4), whole response
    if $armed$ **and** $s \geq \theta$ **then** ▷ Trigger fires
      $armed \leftarrow$ **false**; $n \leftarrow n + 1$
      if $n = 1$ **then** ▷ stage 1: review
        $K \leftarrow E.\textsc{Review}(K, g, \text{episode}, y_t)$
      else ▷ stage 2: withdraw
        $K \leftarrow \varnothing$
      if K changed **then** $y_t \leftarrow \pi\big(x_t(K)\big)$
    else if not $armed$ **and** $s < \theta$ **then**
      $armed \leftarrow$ **true** ▷ re-arm
  execute the action in y_t; **stop** if the episode ends

the pointwise cross-mutual information (P-CXMI) of token $y_{t,i}$ with the block as context (Fernandes et al., 2021, 2023), and averages it over the whole response, reasoning included:

$$S_{\mathrm{lr}}(t) = \frac{1}{|y_t|} \sum_{i=1}^{|y_t|} c_{t,i}. \tag{4}$$

Writing $\pi_B = \pi(\cdot \mid x_t(B))$, the sum $|y_t|\, S_{\mathrm{lr}}(t)$ estimates $\mathrm{KL}(\pi_{K_t} \,\|\, \pi_{\varnothing})$, how far the block moves the agent's response distribution (exactly so only for untempered sampling; App. C.1). The average covers the reasoning because a thinking agent settles its action there: over the action tokens alone the ratio is almost always zero (median 0.00, against 0.36 over the whole response; App. C.3). Averaging per token lets one threshold serve drafts of any length; shorter drafts score higher, but with log length regressed out the score separates the harmed development games as well (AUROC 0.78 against 0.80), and it does not track quoted skill titles (App. C.2). Scoring takes two prefills per step and no generation.

Influence and misfit. S_{lr} measures how strongly the block drives the agent, not whether it fits. A high score marks where a skill-induced wrong choice can occur, not where one does: the Trigger is a gate, not a failure detector, since helpful skills raise S_{lr} too, and the two stages decide between them.

Firing rule. Consecutive high scores count as one event, so the Trigger carries an armed state α_t, with $\alpha_{t_0} = 1$:

$$f_t = \alpha_t\, \mathbb{I}\big[S_{\mathrm{lr}}(t) \geq \theta\big], \qquad \alpha_{t+1} = \mathbb{I}\big[S_{\mathrm{lr}}(t) < \theta\big], \tag{5}$$

where $f_t = 1$ is a firing; steps that are not scored (before t_0, after a withdrawal, or without an action in the draft) leave α unchanged, and $n_t = \sum_{t' \leq t} f_{t'}$ counts the firings. Re-arming only below θ also gives a revised block the chance to take effect before the second stage.

Threshold. As S_{lr} measures influence, not misfit, θ is set as an intervention budget, without outcome labels: a firing rate rather than a score value, because the scale of S_{lr} differs across backbones ($\theta = 1.1423$ for Qwen3-14B by the same rule; App. D.3), and 1.5 firings per episode keeps interventions rare while letting some episodes reach the second stage. With Θ the scores of an observation run of the Skill arm on the development games (every step scored, no intervention; 138 episodes) and $\bar{N}(\theta')$ its mean firings per episode at θ', we set before any test run

$$\theta = \min\{\theta' \in \Theta : \bar{N}(\theta'') \leq 1.5 \ \ \forall\, \theta'' \geq \theta'\}, \tag{6}$$

with $\theta'' \in \Theta$, which gives $\theta = 0.9387$, reached by 5.9% of the scored steps; the condition on all larger θ'' is needed because $\bar{N}$ is not monotone (App. C.3).

4.2 Two-Stage Intervention

The intervention escalates with the number of firings because withdrawal also removes the skills' help (§3.2): the first firing gets the change that keeps most of the block, and the second, the change that removes it. With E the Editor and f_t, n_t from Eq. (5), the block for step t becomes

$$K_t^{+} = \begin{cases} E(K_t, h_t) & f_t = 1,\ n_t = 1 \text{ (revise)}, \\ \varnothing & f_t = 1,\ n_t = 2 \text{ (withdraw)}, \\ K_t & \text{otherwise}, \end{cases} \tag{7}$$

where h_t is the Editor's view of the episode (Stage 1), and $K_{t+1} = K_t^{+}$. If $K_t^{+} \neq K_t$, step t is regenerated, $y_t \sim \pi(\cdot \mid x_t(K_t^{+}))$, with a new seed, and the regenerated action is executed. The second firing counts whether or not the review changed the block, since a block that drives the agent again after its review is one the review could not fix; an episode thus has at most one Editor review, one withdrawal and two regenerated steps.

Method	ALFWorld: success (%)							SRA-Bench ToolQA: accuracy (%)						
	Pick	Heat	Cool	Clean	Look	PickTwo	All	Flt.	Cof.	Yelp	Abnb.	Agd.	DBLP	All
Qwen3-8B (thinking)														
Baselines														
NoSkill	45.8	39.1	66.7	29.0	50.0	29.4	42.5	14.0	38.0	48.0	22.0	16.0	20.0	26.3
Skill (BM25 top-5)	37.5	43.5	52.4	61.3	55.6	5.9	44.8	54.0	58.0	50.0	40.0	8.0	20.0	38.3
+ prompting	50.0	43.5	61.9	61.3	55.6	11.8	49.3	52.0	56.0	50.0	40.0	6.0	26.0	38.3
SkillAligner	66.7	22.7	42.9	58.1	50.0	29.4	46.6	50.0	40.0	52.0	32.0	8.0	10.0	32.0
Planning‡	51.4	31.9	57.1	59.1	51.9	7.8	45.3	40.0	50.0	42.0	36.0	8.0	20.0	32.7
Ours														
TRIM	66.7	60.9	81.0	64.5	55.6	41.2	**62.7**	60.0	68.0	72.0	44.0	8.0	28.0	46.7
Qwen3-14B (thinking)														
Baselines														
NoSkill	72.2	39.1	44.4	40.9	50.0	5.9	43.5	36.0	44.0	56.0	36.0	10.0	30.0	35.3
Skill (BM25 top-5)	68.1	27.5	49.2	64.5	68.5	5.9	49.5	50.0	72.0	40.0	32.0	6.0	32.0	38.7
+ prompting	68.1	30.4	50.8	60.2	70.4	3.9	49.3	52.0	70.0	40.0	28.0	8.0	20.0	36.3
SkillAligner	76.4	24.6	47.6	64.5	61.1	19.6	51.0	54.0	52.0	52.0	42.0	0.0	14.0	35.7
Planning	65.3	39.1	65.1	65.6	74.1	3.9	54.2	46.0	54.0	40.0	20.0	10.0	22.0	32.0
Ours														
TRIM	83.3	40.6	76.2	74.2	79.6	25.5	**64.9**	62.0	74.0	68.0	40.0	8.0	30.0	47.0

Table 2: Main results with Qwen3 in thinking mode. ALFWorld: success on the 134 valid_unseen games per task type (games: 24, 23, 21, 31, 18, 17; SkillAligner finished 133 with Qwen3-8B), in the run that shares the baselines' seeds (TRIM: run 1; its three-run rate is in §5.2); Qwen3-14B rows: three-run means; ‡Qwen3-8B Planning: three-run mean (runs: 41.8, 45.5, 48.5). ToolQA: accuracy on 300 questions, 50 per domain (Flt. Flights, Cof. Coffee, Abnb. Airbnb, Agd. Agenda). Counts: App. D.1. Red: target values for the corrected re-runs (internal draft; NUMERICAL_REGISTRY.md). –: not defined. Controls: App. Table 11.

Stage 1: Editor. At the first firing, the Editor, the agent's own model with thinking off, reads h_t: the task, the first observation and the last eight steps, the draft with the end of its reasoning, and the block (App. C.4). It gives each item κ_j a verdict $v_j \in \{\text{KEEP}, \text{WITHHOLD}, \text{REPLACE}(\kappa'_j)\}$, and the edited block is

$$E(K_t, h_t) = \left[\rho(\kappa_j, v_j)\right]_{j:\, v_j \neq \text{WITHHOLD}} \oplus u, \quad (8)$$

where $\rho(\kappa, \text{KEEP}) = \kappa$, $\rho(\kappa, \text{REPLACE}(\kappa')) = \kappa'$, and u is an optional instruction for a step the task needs and no item covers. KEEP is the default; an item is withheld when following it conflicts with the task or the observations, and replaced when its idea serves the task but a detail is wrong. What the task needs is fixed first, in a separate call that sees only the task sentence, so a misled plan cannot argue for keeping an item; routines that mislead over many steps are left to Stage 2. The Editor never sees S_{lr} or any outcome.

Stage 2: Withdrawal. At the second firing the Editor is not called: the block is withdrawn for the rest of the episode, $K_{t'} = \varnothing$ for all $t' \geq t$, step t is regenerated from $x_t(\varnothing)$, and the Trigger stops scoring.

5 Experiments

5.1 Experimental Setup

We evaluate on the 134 unseen games (`valid_unseen`) of ALFWorld (Shridhar et al., 2021) in the SkillRL harness (Xia et al., 2026), whose two-step history we keep for comparability with SkillRL, and on 300 questions of SRA-Bench ToolQA (Su et al., 2026; Zhuang et al., 2023), with Qwen3-8B and Qwen3-14B (Yang et al., 2025) in thinking mode. The block holds the five SkillBank skills ranked highest by BM25 (Robertson and Zaragoza, 2009). We report success rates (Table 2) and compare arms per game with exact sign tests and 95% bootstrap intervals (App. D.4). The baselines, prompting, SkillAligner (Li et al., 2026a) and Planning (Wang et al., 2026a) (Table 3), differ from TRIM only in how the skills reach the prompt, and two timing controls run TRIM's stages at fixed or random steps.

5.2 Main Results

Overall. With Qwen3-8B, retrieved skills raise success from 42.5% to 44.8%, prompting reaches 49.3% and SkillAligner 46.6%, while TRIM reaches 62.7% in the run that shares the baselines' seeds and 59.5% over its three runs. Adapting the skills at injection thus adds 4.5 and 1.8 points

Arm	Skill block in the prompt	Changed
NoSkill	none	–
Skill	top five by BM25 on the goal	never
+ prompting	Skill's, plus one line	never
SkillAligner	rewritten into a guide	at injection
Planning	retrieved per plan step	at injection
One-time edit	reviewed by the Editor	step 2
Random steps	reviewed, then withdrawn	random steps
TRIM	reviewed, then withdrawn	its firings

Table 3: Arms. All share the agent, the harness and the five skills retrieved for each game, and differ only in how the block reaches the prompt and when it changes. The two timing controls run TRIM's Editor once at step 2 or both of its stages at random steps.

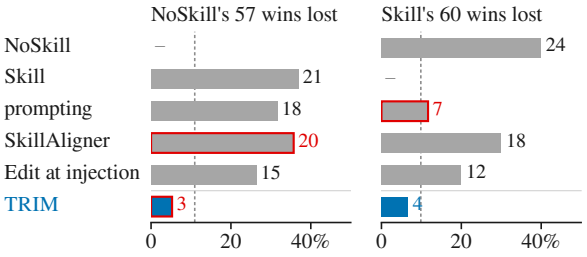


Figure 3: Wins of NoSkill (left) and of Skill (right) that each arm loses on the 134 test games (Qwen3-8B, thinking, run 1; SkillAligner: 133 games). Edit at injection (PRE): TRIM's Editor run once, when the block is injected at step 2. Dashed: a second run of the same arm (development games).

to the retrieved skills, whereas revising and withdrawing them after injection adds 17.9 points in that run. Its per-game success rate is 14.7 points above Skill's (95% CI [8.7, 20.9]) and 16.9 above NoSkill's ([10.0, 23.9]; App. Figure 7); its first run is ahead of Skill (28:4 games, $p < 0.001$) and NoSkill (30:3, $p < 0.001$) after a Holm adjustment (Holm, 1979).

Where the gains come from. The gains follow the misfit of §3. Where the skills hurt, TRIM recovers past the agent without skills: Cool rises from 52.4% with Skill to 81.0% (NoSkill 66.7%) and Pick from 37.5% to 66.7% (45.8%). Where they help, it adds to their gain on Clean (64.5%, against 61.3% with Skill and 29.0% without) and on Heat (60.9%, against 43.5% and 39.1%).

Other backbone and benchmark. With Qwen3-14B (three runs), the retrieved skills raise success from 43.5% to 49.5%, prompting reaches 49.3% and SkillAligner 51.0%, and TRIM 64.9%, ahead of every baseline in per-game sign tests ($p < 0.005$, also after Holm); on ToolQA TRIM answers 46.7% and 47.0% of the questions with the two backbones, ahead of every arm ($p < 0.001$; App. E.1).

Lost wins and cost. Deciding about the skills once, at injection, gives up wins on one side or both (Figure 3): keeping them loses 21 of NoSkill's 57 wins, dropping them loses 24 of Skill's 60, prompting still loses 18 of NoSkill's wins and 7 of Skill's, and SkillAligner, which rewrites the skills at injection, 20 and 18. Even TRIM's Editor, run once when the block is injected (PRE; App. E.2), loses 15 and 12. TRIM, which revises and withdraws the block after injection, loses 3 and 4, the fewest of all arms with skills; a second run of NoSkill or Skill itself loses 7 of 64 and 7 of 71 wins on the development games. On the 26 games all these arms solve, TRIM takes 9.3 actions (Skill 8.2, NoSkill 11.5), and it generates 31.0k tokens per episode with Editor calls, against 32.0k for Skill and 35.8k for SkillAligner (App. Table 10).

5.3 Ablation Study

Table 4 changes one part of TRIM at a time on the development games, counting wins among the 15 games the skills harm and the 20 that need them; each stage covers one side. Without the withdrawal, 3 harmed games are won instead of 10 and success falls from 61.9% to 49.3%; at the 42 states where TRIM withdrew, continuations without the block win 35.7% against 23.8% with it (+11.9 points, [1.2, 22.6]; Figure 4). Without the edit, withdrawing at the first firing wins less often (50.7%; 18:3 games, $p = 0.001$) and keeps only 10 of the 20 games that need the skills, against 18 of 19, so withdrawal must be selective (§6). The signal times the withdrawal: at random steps 4 harmed games are won, and 5 with both stages random (App. E.3). TRIM also wins more test episodes than a one-time edit at step 2 or the same two stages at random steps (239 against 192 and 204 of 402, $p < 0.001$), and withdrawing at the signal's state helps more than at nearby or random states (App. E.2 and E.4).

6 Analysis

Counts come from the first test run unless more runs or the development games are named.

Firing statistics. TRIM fires in 81 of the 134 test episodes of run 1, first at a median step of 4 (interquartile range 2–10); 64 of its 81 reviews change the block, and 34 episodes reach a second

Variant	%	Harmed	Needed
TRIM (edit → withdraw)	**61.9**	**10**/15	**18**/19
Stages			
Edit only	49.3	3/15	14/20
Withdraw only	50.7	6/15	10/20
Regenerate → withdraw	48.6	5/15	15/20
Random timing			
Withdrawal	51.4	4/15	15/20
Both stages	52.1	5/15	17/20
No intervention (Skill)	50.0	(0/15)	(20/20)

Table 4: Components of TRIM on the 140 development games (Qwen3-8B, thinking; one run each, same seeds; %: of finished games). Withdraw only: at the first firing, no edit; random timing: random steps, not the signal. Harmed / Needed: wins among the 15 games the skills harm and the 20 that need them, set by Skill's and NoSkill's runs (hence Skill's values). More: App. E.3, Table 12.

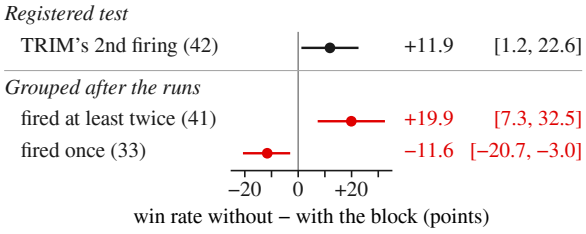


Figure 4: Withdrawing the block on restored development states (Qwen3-8B, thinking): win rate without the block minus with it, in points, with 95% bootstrap CIs. Top: the 42 states where TRIM withdrew, against keeping the edited block. Bottom: episodes grouped by whether the signal had fired at least twice or once by step 40, averaging three restored states each; the groups differ by 31.5 points, [16.2, 46.8]. Within an episode, withdrawing at the signal's state helps more than at the nearest or a random state without a firing (+12.8 and +16.2 points; App. Table 13).

firing and withdraw the block, at a median step of 15 (9–20). Over the three runs, the first firing comes at a median step of 4 in every run (App. F.1).

Repeated firing and withdrawal. On the development observation run (no intervention), the re-armed Trigger fires 3.07 times per episode in the 15 games the skills harm, against 1.45 in the 20 games that need them and 0.59 in the 44 that every run wins. It fires at least twice in 11 of the harmed games, 8 of those that need the skills and 2 of the 44.[Draft note: step-level link (firing step = step of the skill-induced wrong choice) not yet measured; current evidence is per episode] Firing counts grow with episode length, so we test withdrawal at fixed states: we restored 74 development episodes in which the signal fired, each at three of its states, and continued every state twice with the block and twice without it (App. E.4). In the 41 episodes where the signal fired at least twice, withdrawal raises the win rate by 19.9 points; in the 33 where it fired once, it lowers it by 11.6 points (difference 31.5 points, 95% CI [16.2, 46.8]; Figure 4).

Gains and losses of withdrawal. Split by how Skill and NoSkill fared on the game, withdrawal at the signal's state raises the win rate by 57.7 points in the games the skills harm and lowers it by 34.6 points in the games that need them (13 episodes each; App. Table 13), the same trade-off as in the same-state study (§3.2); withdrawing only when the Trigger fires again after the review keeps the skills where they help (§5.2).

Effect of each stage. The edit removes procedures written for other tasks: on the test games, its 81 first reviews withhold or replace 57 of the 79 items written for another task type, and in edit-only development arms, in 109 of 129 drafts that planned a procedure the task does not ask for, the regenerated reasoning no longer did. A search routine instead spans several items and plays out over many steps, and the withdrawal ends it: in the game of Figure 2 the agent reaches the countertop seven steps after the withdrawal, and in the game of Figure 8 it leaves the cabinets at the next step.

7 Conclusion

A skill that fits the goal can still misfit the episode: with a reasoning agent on ALFWorld, the retrieved skills alone can make the agent fail from states where it would otherwise succeed, mostly by locking it into a search routine or another task's procedure. TRIM revises the injected skills when they drive the agent's drafted response and withdraws them if they keep driving it. In three test runs with Qwen3-8B, it wins 79.7 of 134 games on average, 19.7 more than the retrieved skills, and by withdrawing the skills only after a revision fails it loses 4 of their 60 test wins in the first run, against 24 when they are dropped, so skills should be withdrawn selectively rather than dropped. It also leads with Qwen3-14B and on ToolQA. Separating the misfits that skills cause from failures that happen anyway remains open. Skills should be revisited after injection, where they act.

Limitations

We evaluate TRIM with one model family, Qwen3, in thinking mode, on ALFWorld with one retrieved skill library and on SRA-Bench ToolQA; other environments, skill libraries written by people and other model families remain to be tested. The Trigger reads prompt log-probabilities, which many closed APIs do not expose, and scoring the drafted response with and without the block adds two prefills per step, 8.5% of an episode's wall-clock time with Qwen3-8B (App. D.3). TRIM revises or withdraws the retrieved block but does not write new skills, so failures whose procedure is missing from the library are outside its scope; the Editor is the agent's own checkpoint, and a stronger Editor may revise blocks more precisely; θ follows a firing-rate target (1.5 per episode) that we did not vary.

References

Anthropic. 2025. Equipping agents for the real world with Agent Skills. Anthropic Engineering Blog. Written by Barry Zhang, Keith Lazuka, and Mahesh Murag; published October 16, 2025.

Chishui Chen, Jiaye Lin, Te Sun, Yi Yang, Junxi Wang, Cong Qin, Yangen Hu, Lu Pan, and Ke Zeng. 2026. Skill or skip? Learning selective skill invocation in agentic tasks via dual-granularity preference learning. *Preprint*, arXiv:2606.00510.

Benjamin Cohen-Wang, Harshay Shah, Kristian Georgiev, and Aleksander Mądry. 2024. ContextCite: Attributing model generation to context. In *Advances in Neural Information Processing Systems*, volume 37, pages 95764–95807. Curran Associates, Inc.

Gen Dong, Yanjie Gao, Liqun Li, Tianyin Xu, Yu Hua, and Fan Yang. 2026. Agent skills can be harmful: An empirical study of skill-induced failures in LLM agents. *Preprint*, arXiv:2608.11888.

Rotem Dror, Gili Baumer, Segev Shlomov, and Roi Reichart. 2018. The hitchhiker's guide to testing statistical significance in natural language processing. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1383–1392, Melbourne, Australia. Association for Computational Linguistics.

Patrick Fernandes, Kayo Yin, Emmy Liu, André Martins, and Graham Neubig. 2023. When does translation require context? A data-driven, multilingual exploration. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 606–626, Toronto, Canada. Association for Computational Linguistics.

Patrick Fernandes, Kayo Yin, Graham Neubig, and André F. T. Martins. 2021. Measuring and increasing context usage in context-aware machine translation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 6467–6478, Online. Association for Computational Linguistics.

Yao Fu, Dong-Ki Kim, Jaekyeom Kim, Sungryull Sohn, Lajanugen Logeswaran, Kyunghoon Bae, and Honglak Lee. 2024. AutoGuide: Automated generation and selection of context-aware guidelines for large language model agents. In *Advances in Neural Information Processing Systems*, volume 37, pages 119919–119948. Curran Associates, Inc.

Tingxu Han, Yi Zhang, Wei Song, Chunrong Fang, Zhenyu Chen, Youcheng Sun, and Lijie Hu. 2026. SWE-Skills-Bench: Do agent skills actually help in real-world software engineering? *Preprint*, arXiv:2603.15401.

Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations*.

Zhengbao Jiang, Frank Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7969–7992, Singapore. Association for Computational Linguistics.

Philipp Koehn. 2004. Statistical significance tests for machine translation evaluation. In *Proceedings of the 2004 Conference on Empirical Methods in Natural Language Processing*, pages 388–395, Barcelona, Spain. Association for Computational Linguistics.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626. Association for Computing Machinery.

Qinfeng Li, Dalin He, Yuntai Bao, Ying Yang, Ruoxi Chen, Xinyan Yu, Lizhou Liang, Ge Su, Wenqi Zhang, and Xuhong Zhang. 2026a. SkillAligner: Treating retrieved skills as adaptable drafts at execution time. *Preprint*, arXiv:2608.06880.

Xiangyi Li, Yimin Liu, Wenbo Chen, Bingran You, Zonglin Di, Yifeng He, Shenghan Zheng, Kyoung Whan Choe, Jiankai Sun, Shuyi Wang, Chujun

Tao, Binxu Li, Xuandong Zhao, Hejia Geng, Xiaojun Wu, Junwei Zhou, Xiaokun Chen, Hanwen Xing, Yubo Li, and 59 others. 2026b. SkillsBench: Benchmarking how well agent skills work across diverse tasks. *Preprint*, arXiv:2602.12670.

Yanchao Li, Wanhao Liu, Ben Gao, Jiaqing Xie, Zhehong Ai, Na Zou, Yuqiang Li, and Tianfan Fu. 2026c. SkillsInjector: Dynamic skill context construction for LLM agents. *Preprint*, arXiv:2605.29794.

Bohan Lin, Hejia Geng, Xinyi Xie, Heng Zhou, Qinghua Xing, Bo Liu, Chen Zhang, and Yudong Zhang. 2026. Emotion2Skill: Model-internal emotion signals for adaptive skill selection and evolution. *Preprint*, arXiv:2608.09248.

Yujian Liu, Jiabao Ji, Li An, Tommi Jaakkola, Yang Zhang, and Shiyu Chang. 2026a. How well do agentic skills work in the wild: Benchmarking LLM skill usage in realistic settings. In *Third Conference on Language Modeling*.

Yuxuan Liu, Zhaochen Su, Lingyun Xie, Yuhao Zhang, Qing Zong, Jiahe Guo, Zhongwei Xie, Yiyan Ji, Yauwai Yim, Hongyu Luo, Xiyu Ren, Ruan Chenyu, Haoran Li, and Yangqiu Song. 2026b. SkillRevise: Improving LLM-authored agent skills via trace-conditioned skill revision. *Preprint*, arXiv:2606.01139.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, volume 36, pages 46534–46594. Curran Associates, Inc.

Siru Ouyang, Jun Yan, I-Hung Hsu, Yanfei Chen, Ke Jiang, Zifeng Wang, Rujun Han, Long T. Le, Samira Daruki, Xiangru Tang, Vishy Tirumalashetty, George Lee, Mahsan Rofouei, Hangfei Lin, Jiawei Han, Chen-Yu Lee, and Tomas Pfister. 2026. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *The Fourteenth International Conference on Learning Representations*.

Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389.

Gabriele Sarti, Grzegorz Chrupała, Malvina Nissim, and Arianna Bisazza. 2024. Quantifying the plausibility of context reliance in neural machine translation. In *The Twelfth International Conference on Learning Representations*, pages 25526–25554.

Freda Shi, Xinyun Chen, Kanishka Misra, Nathan Scales, David Dohan, Ed H. Chi, Nathanael Schärli, and Denny Zhou. 2023. Large language models can be easily distracted by irrelevant context. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 31210–31227. PMLR.

Weijia Shi, Xiaochuang Han, Mike Lewis, Yulia Tsvetkov, Luke Zettlemoyer, and Wen-tau Yih. 2024. Trusting your evidence: Hallucinate less with context-aware decoding. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 2: Short Papers)*, pages 783–791, Mexico City, Mexico. Association for Computational Linguistics.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, pages 8634–8652. Curran Associates, Inc.

Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. 2021. ALFWorld: Aligning text and embodied environments for interactive learning. In *The Ninth International Conference on Learning Representations*.

Weihang Su, Jianming Long, Qingyao Ai, Qiaozhi He, Yichen Tang, Changyue Wang, Yiteng Tu, Yingbo Wang, and Yiqun Liu. 2026. Skill retrieval augmentation for agentic AI. *Preprint*, arXiv:2604.24594.

Weihang Su, Yichen Tang, Qingyao Ai, Zhijing Wu, and Yiqun Liu. 2024. DRAGIN: Dynamic retrieval augmented generation based on the real-time information needs of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12991–13013, Bangkok, Thailand. Association for Computational Linguistics.

Haotian Sun, Yuchen Zhuang, Lingkai Kong, Bo Dai, and Chao Zhang. 2023. AdaPlanner: Adaptive planning from feedback with language models. In *Advances in Neural Information Processing Systems*, volume 36, pages 58202–58245. Curran Associates, Inc.

Darshan Tank and Baran Nama. 2026. The regression tax: Decomposing why skills help and hurt LLM agents. *Preprint*, arXiv:2607.22520.

Songjun Tu, Chengdong Xu, Qichao Zhang, Yiwen Ma, Yaocheng Zhang, Linjing Li, Dong Li, Xiangyuan Lan, and Dongbin Zhao. 2026. UCOB: Learning to utilize and evolve agentic skills via credit-aware on-policy bidirectional self-distillation. *Preprint*, arXiv:2606.29502.

Chenxi Wang, Zhuoyun Yu, Xin Xie, Wuguannan Yao, Runnan Fang, Shuofei Qiao, Kexin Cao, Guozhou Zheng, Xiang Qi, Peng Zhang, and Shumin Deng. 2026a. SkillX: Automatically constructing skill knowledge bases for agents. *Preprint*, arXiv:2604.04804.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*.

Hao Wang, Guozhi Wang, Han Xiao, Yufeng Zhou, Yue Pan, Jichao Wang, Ke Xu, Yafei Wen, Xiaohu Ruan, Xiaoxin Chen, and Honggang Qi. 2026b. Skill-SD: Skill-conditioned self-distillation for multi-turn LLM agents. *Preprint*, arXiv:2604.10674.

Yixuan Wang, Yiyang Zhou, Yiming Liang, Congyu Zhang, Fuxiao Liu, Jiawei Zhou, and Huaxiu Yao. 2026c. Not all skills help: Measuring and repairing agent knowledge. *Preprint*, arXiv:2606.15390.

Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. 2025. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911. PMLR.

Peng Xia, Jianwen Chen, Hanyang Wang, Jiaqi Liu, Kaide Zeng, Yu Wang, Siwei Han, Yiyang Zhou, Xujiang Zhao, Haifeng Chen, Zeyu Zheng, Cihang Xie, and Huaxiu Yao. 2026. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning. *Preprint*, arXiv:2602.08234.

An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *Preprint*, arXiv:2505.09388.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*.

Jiayi Yuan, Hao Li, Xinheng Ding, Wenya Xie, Yu-Jhe Li, Wentian Zhao, Kun Wan, Jing Shi, Xia Hu, and Zirui Liu. 2025. Understanding and mitigating numerical sources of nondeterminism in LLM inference. In *Advances in Neural Information Processing Systems*, volume 38, pages 169819–169851. Curran Associates, Inc.

Kaixing Zhang, Changming Li, Yingdong Shi, Zheng Zhang, Kaitao Song, Wenjie Shi, Jingang Wang, and Kan Ren. 2026a. Rep2Skill: Representation-guided skill self-evolution for LLM agents. *Preprint*, arXiv:2609.39149.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Y. Zou, and Kunle Olukotun. 2026b. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations*.

Wenyuan Zhang, Xinghua Zhang, Haiyang Yu, Shuaiyi Nie, Bingli Wu, Juwei Yue, Tingwen Liu, and Yongbin Li. 2026c. ExpSeek: Self-triggered experience seeking for web agents. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 29255–29283, San Diego, California, United States. Association for Computational Linguistics.

Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. 2024. ExpeL: LLM agents are experiential learners. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(17):19632–19642.

Ruiwen Zhou, Yingxuan Yang, Muning Wen, Ying Wen, Wenhao Wang, Chunling Xi, Guoqiang Xu, Yong Yu, and Weinan Zhang. 2024. TRAD: Enhancing LLM agents with step-wise thought retrieval and aligned decision. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 3–13. Association for Computing Machinery.

Yuchen Zhuang, Yue Yu, Kuan Wang, Haotian Sun, and Chao Zhang. 2023. ToolQA: A dataset for LLM question answering with external tools. In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 36, pages 50117–50143. Curran Associates, Inc.

A Extended Related Work

This section covers the work that §2 points to: methods that revise guidance during an episode, and other uses of the with/without-context contrast that the Trigger computes.

Revision during an episode. Irrelevant context misleads LLMs (Shi et al., 2023). During an episode, exemplars are re-retrieved (Zhou et al., 2024), guidelines selected (Fu et al., 2024) and plans revised (Sun et al., 2023). These methods change exemplars, guidelines or plans; TRIM keeps the executor unchanged and revises, then withdraws, the injected skill block when the Trigger fires. Self-correction runs after every output or after a failed attempt (§2); the Editor of TRIM runs only at the Trigger's first firing, and a one-time edit at injection and the two stages at random steps both win fewer games than TRIM (App. E.2).

Contrasts with and without context. Agents seek experience when their step-level entropy is high (Zhang et al., 2026c); the Trigger instead asks whether the injected block drives the response, and App. C.3 compares it with an entropy-based candidate. The with/without-context contrast also attributes generations to context (Cohen-Wang et al., 2024), reweights decoding (Shi et al., 2024)

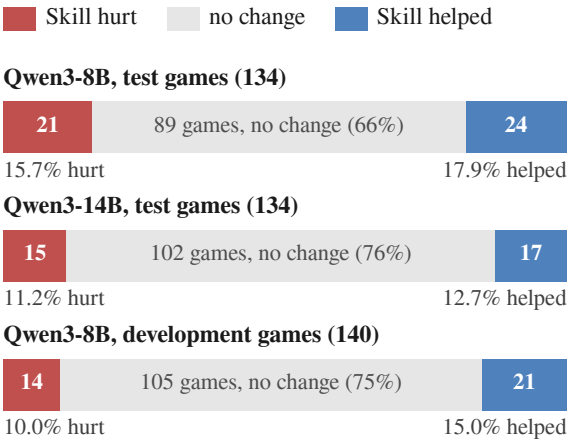


Figure 5: Outcome changes with and without the retrieved skills (Qwen3, thinking; one run per arm). Hurt: NoSkill wins the game and Skill loses it; helped: the reverse. The block changes 45 of the 134 test games with Qwen3-8B (21 hurt, 24 helped) and 32 with Qwen3-14B (15, 17); on the 140 development games it changes 35, against 19 between two NoSkill runs.

State	Type	k	Keep	Edit	No block
Harmed (Skill lost, NoSkill won)					
S01	Heat	3	0/3	1/3	3/3
S02	Clean	3	0/3	2/3	1/3
S03	Cool	3	0/3	2/3	1/3
S04	Heat	30	0/3	1/3	1/3
S05	Heat	8	0/3	2/3	3/3
S06	Cool	3	0/3	0/3	2/3
Harmed, total			0/18	8/18	11/18
Helped (Skill won, NoSkill lost)					
S07	Heat	45	2/3	2/3	0/3
S08	Clean	11	3/3	3/3	3/3
S09	Clean	6	2/3	1/3	3/3
Helped, total			7/9	6/9	6/9
Both lost					
S10–S12			0/9	0/9	0/9

Table 5: Same-state continuations on development states (Qwen3-8B, thinking): wins of three continuations per branch from the restored state at step k. Without the block the agent wins from all six harmed states; with the unchanged block, from none.

and trains skill-conditioned students (Wang et al., 2026b); TRIM uses it only to time an intervention at inference.

B Motivating Analysis Details

This section details the analyses of §3; Figure 5 draws the outcome changes of Table 1a. Restoring the same state and continuing it with the block kept, edited or removed separates the effect of the block from the trajectory it produced, which a comparison of independent runs cannot do; the rerun variation of App. B.3 is the baseline against which the skills' effect on outcomes is measured.

B.1 Same-State Continuations

We sampled 12 games of a development run (seed fixed in advance), stratified by its outcomes: six that Skill lost and NoSkill won (harmed), three that Skill won and NoSkill lost (helped) and three that both lost. In each, we restored the Skill episode at step k, one step after the agent first picked up an object, or at step 3 if it never did, replaying the logged actions and checking that every restored prompt equals the logged one. From that state, each branch regenerates step k and continues to the 50-step limit three times (seeds fixed per branch), with no further intervention: A keeps the block, B uses the block after one review by the Editor of §4.2 without the reasoning excerpt, and N removes it (the no-skill prompt). Table 5 gives every state; four of the six harmed episodes never pick up an object, so their states are restored at step 3. From every harmed state, the agent without the block wins at least once and the agent with the unchanged block never does (6 of 6 states; exact sign test, $p = 0.031$); the edited block wins from five of the six. From the helped states, removing the block costs one of nine wins (7 against 6).

B.2 Failure Patterns

Of the 15 development games that the skills harm (Skill loses both of its runs, NoSkill wins at least one of its two), 13 follow one of two patterns, both of which take effect only after injection: a search tunnel (9 games) and a procedure written for another task type (4 games). **Search tunnels (9 games).** In both Skill runs the agent never observed the target, and no Skill episode visited the place where NoSkill picked it up; 91% and 98% of Skill's moves went to closed containers, against 59% and 63% for NoSkill. In six of them the reasoning cites "Systematic Object Hunt" in 37 to 48 of the 49 steps. **Procedures for another task (4 games).** BM25 attached a state-change item for another task type through a shared word, such as "Enforce Cooling Before Placement" in heating tasks whose destination is the fridge, or "Open Then Heat" in cooling tasks whose destination is the microwave. Over all development games, heating games with the cooling item are won in 0 of 8 by both Skill runs and in 2 and 4 by NoSkill; cooling games with the heating

item in 0 and 1 of 6 by Skill and 4 and 4 by NoSkill. Over these 14 games with a misattached item, Skill wins 1 of 28 episodes and NoSkill 14, while Skill wins 5 and 4 of the 7 heating games without the cooling item. **Unrelated (2 games).** One agent took a failed pick-up for a success, and one read "cool" as already cold, as NoSkill also did once. The Editor's verdicts on these items are analyzed in App. F.2.

B.3 Run-to-Run Noise

The skill block changes 35 of the 140 development outcomes, against 19 between two runs of NoSkill and 13 of 139 between two runs of Skill (11% on average); we therefore compare arms game by game (App. D.4). With the same prompts and seeds, batched serving makes runs diverge (Yuan et al., 2025): 19–28% of games already differ in the first step. The difference in wins between two runs of 140 games has a standard deviation of 3.7.

C Method Details

This section gives the implementation of TRIM used in every test run. Table 6 lists all settings; each was fixed before any test run, on the 140 `valid_seen` development games for ALFWorld and on 120 development questions for ToolQA. The threshold, one per backbone and benchmark, is the only value fitted to data; the signal and the intervention were each chosen among candidates by a rule registered before their results (App. C.3 and E.3).

C.1 Implementation Details

TRIM runs inside SkillRL's released ALFWorld agent loop (Xia et al., 2026), ported string for string, so every arm sees the same prompt, history and parser and differs only in the block that reaches it.

Harness. The port covers the prompt templates, the history format (the last two observation–action pairs), the admissible-action list (without `help`) and the action parser. We keep the two-step history for comparability with SkillRL's protocol, with which its SkillBank was released. Every arm shares the loop, so the misfit we report, such as the search tunnels of App. B.2, is a difference between arms within it. Step 1 uses SkillRL's no-history template without skills; from step 2 on, the block appears under `## Retrieved Relevant Experience`. The parser takes the lower-cased text inside the first `<action>...</action>` of the answer, after the reasoning; if a tag is missing, the last 30 characters of the output are sent to the environment, which answers "Nothing happens.". An episode ends with success or after 50 actions.

Decoding. The agent is Qwen3-8B or Qwen3-14B (Yang et al., 2025) with thinking enabled in its chat template, sampled with temperature 0.6, top-p 0.95 and top-k 20 (no penalties) and at most 4,096 output tokens per response, and served with vLLM (Kwon et al., 2023) and its reasoning parser, which returns the reasoning and the answer separately. Each request carries a seed derived from the game, the step and the request kind (draft or regeneration), so all arms draw the same seed for the same game and step.

Retrieval. The skill bank is SkillRL's released SkillBank with 44 retrievable skills (12 general, 32 task-specific; its 11 "common mistakes" form a fixed section and are not retrieved). Each skill is indexed as "title. principle. when_to_apply" and the query is the goal sentence. We rank with Okapi BM25 (Robertson and Zaragoza, 2009) (k_1=1.5, b=0.75, lower-cased alphanumeric tokens, no stemming or stop-word removal) and keep the top five. General skills are rendered as `- **title**: principle` under `### General Principles` and task skills as the same line plus an indented `_Apply when: . . . _` line under `### Task-Relevant Skills`.

C.2 Computation of S_{lr}

S_{lr} needs two prefills of the already drafted response and no generation: the skill-off term rescores the same tokens under the no-skill prompt and never regenerates the response. The drafted response is rebuilt as `<think>`, the reasoning, `</think>` and the answer, and tokenized once. Each of the two prompts, $x_t(K)$ and $x_t(\varnothing)$, is chat-templated with thinking enabled and followed by these token ids, and one prefill returns the log-probability of every response token (temperature 0, one generated token, top-5 prompt log-probabilities). S_{lr} is the mean difference over all response tokens, in nats per token; a draft without an action in its answer is not scored and cannot fire. App. D.3 reports the cost of scoring.

Length and quoting. S_{lr} averages over tokens so that one threshold serves drafts of any length (141 to 4,062 tokens on the observation run of App. C.3); the sum $|y_t|\, S_{\mathrm{lr}}(t)$ grows with the length of the

reasoning. Over the 139 development games that have strata (App. E.3; 4,229 scored steps), shorter drafts score higher (Pearson -0.64 between S_{lr} and log length), but length does not drive the separation: the residual of S_{lr} after a linear regression on log length separates the eight positive games of App. C.3 as well (AUROC 0.78, against 0.80 for S_{lr}) and, firing at the same share of steps as S_{lr}, fires in the same 13 of the 15 games the skills harm and in 13 of the 20 that need them (12 shared). S_{lr} does not reduce to quoting either: the reasoning names a title of the block at 59% of the scored steps, and a trigger that fires on such mentions (re-armed like the Trigger) fires 5.2 times per episode and in 19 of the 20 games that need the skills, whereas S_{lr} reaches θ at 8% of these steps.

C.3 Signal Selection and Threshold

The ratio covers the whole response because a reasoning agent settles its action while reasoning: over the action tokens alone it is almost always zero on the observation run below (median 0.00, 90th percentile 0.001, against 0.36 and 0.80 over the whole response). The registered selection rule, written before any intervention ran, also picks the whole-response ratio: it is the only registered signal that fires in all eight positive games (AUROC 0.80, 95% interval $[0.67, 0.92]$; Jensen–Shannon 0.75, entropy difference 0.72, last-64-token window 0.70, each firing in seven). An observation run of the Skill arm on the 140 `valid_seen` games logged every candidate at every step without intervening (4,278 scored steps). The four single candidates come from the two prefills of App. C.2: S_{lr} over the whole response, S_{lr} over the last 64 reasoning tokens and the action, the mean Jensen–Shannon divergence of the top-5 next-token distributions, and the mean entropy difference. The rule ranks each candidate by its AUROC, computed on the episode's maximum value, in separating the 8 positive games, which Skill lost in both of its runs and NoSkill won in both (a strict subset of the 15 harmed games of App. B.2), from the other 130 finished games; each candidate's threshold follows Eq. (6).

At θ the Trigger fires in all eight positive games and in 60% of the other games, where the first firing only triggers a review. A permissive first firing is cheap, because it only calls the Editor, whose default verdict is keep; the withdrawal waits for a second firing, which concentrates in the games the skills harm (§6).

Two combinations of the candidates were registered as well, a standardized mean of three signals (AUROC 0.75) and a two-stage gate (0.82), and each fires in seven of the eight positive games. Neither improves development success over the single ratio (66 and 65 against 66 of 140 games with edits only; Table 12), so TRIM uses one ratio and one threshold.

Threshold search. No outcome label enters θ: it fixes an intervention budget, a mean of at most 1.5 firings per episode on the observation run. Θ holds the distinct values of the oriented signal on the 138 observation episodes that had finished when the rule ran; for the whole-response ratio these are 4,181 values, all distinct. The mean number of firings per episode, $\bar{N}$, is not monotone in the threshold: it is 1.0 at the smallest value, where the Trigger fires at the first scored step and never re-arms, exceeds 1.5 from 0.059 to 0.9377 (at most 6.2 firings per episode) and stays at or below 1.5 from 0.9387 on. Eq. (6) therefore takes the smallest value from which $\bar{N} \le 1.5$ holds for all larger candidates, $\theta = 0.938709$ at full precision ($\bar{N} = 1.5$), which 5.9% of the scored steps reach; the smallest value with $\bar{N} \le 1.5$ alone would be the smallest score (0.023), at which the Trigger fires once in every episode. The code finds θ by bisection over the sorted Θ; evaluating $\bar{N}$ at every value of Θ gives the same value.

Other backbones and benchmarks. The same rule sets θ for every backbone and benchmark, each on a development set disjoint from the test set. For Qwen3-14B on ALFWorld it gives $\theta = 1.142321$, which 5.8% of the 4,121 scored steps of its observation run reach, so the bound of 1.5 firings per episode gives nearly the same share of firing steps on both backbones. ToolQA episodes score only 4.4 (Qwen3-8B) and 4.7 (Qwen3-14B) steps on average, too few for the per-episode bound to bind, so there θ keeps ALFWorld's share. It is the smallest value reached by at most 5.891% of the scored steps of an observation run on 120 development questions ($\theta = 0.770$ for Qwen3-8B and 0.758 for Qwen3-14B, reached by 5.86% and 5.71% of the steps; App. D.1). The intervention applied at these firings was chosen by a second registered rule (App. E.3).

C.4 Editor Prompt and Decoding

The Editor's instructions name no action vocabulary or task type: only the opening line and the

Setting	Value
Retrieval	BM25 (k_1=1.5, b=0.75), top five
Injection step	$t_0 = 2$
Signal	S_{lr} over the whole response
Scoring cost	two prefills per scored step
θ, ALFWorld	0.938709 (8B), 1.142321 (14B)
θ, ToolQA	0.770 (8B), 0.758 (14B)
Re-arming	after a step with $S_{\mathrm{lr}} < \theta$
First firing	one Editor review (two calls); regenerate if the block changed
Second firing	withdraw the block for the rest of the episode; regenerate; stop scoring
Editor input	8 executed steps, observations clipped to 300 characters, drafted action, last 1,500 characters of reasoning
Editor decoding	agent's checkpoint, thinking off, temperature 0, seed 0, $\leq$1,200 tokens
Agent decoding	thinking, T 0.6, top-p 0.95, top-k 20, $\leq$4,096 tokens
Episode budget	50 actions (ALFWorld)

Table 6: Hyperparameters of TRIM (8B, 14B: Qwen3-8B, Qwen3-14B). θ is set by the rule of §4.1 (App. C.3) on a development observation run; the signal and the two-stage rule were chosen among registered candidates (App. C.3 and E.3); the remaining values were fixed before any run.

three harness facts describe the environment (Figure 6), and on ToolQA the Editor keeps the NEEDS definition and every review rule word for word, with its own opening line and harness facts. The Editor is the agent's own checkpoint and runs with thinking disabled, temperature 0, seed 0 and at most 1,200 output tokens. One review takes two calls: the first sees only the task sentence and writes NEEDS, which is then fixed; the second returns a verdict for every item. Its input is the view h_t of §4.2: the task, the harness facts, the initial observation and the last eight executed steps (observations clipped to 300 characters), the admissible actions, the drafted action, the last 1,500 characters of the draft's reasoning, the line-numbered block, NEEDS and the current state of each item. Each item receives one verdict (keep, withhold, replace with a corrected instruction, or restore an earlier change), and the Editor can add one instruction; withheld items disappear from the block that the agent sees, replaced items show the corrected instruction, and an added instruction is appended. The Editor never sees S_{lr} or any outcome. TRIM reviews once per episode, so every item is in its original state at the review; the item states and the RESTORE verdict of the prompt only matter for repeated reviews. An ADD line is appended as an instruction unless it reads exactly 'ADD: NONE'.

D Experimental Details

All arms share one agent loop, one seed schedule and one retrieved block per game; this section gives what is needed to reproduce every comparison.

D.1 Benchmarks and Backbones

We run Qwen3 (Yang et al., 2025) in thinking mode, so that the failures that remain expose misfit: on the 140 development games, thinking raises the agent's wins from 30 to 64 without skills and from 37 to 71 with them. Qwen3-8B and Qwen3-14B use Qwen3's recommended sampling (temperature 0.6, top-p 0.95, top-k 20) with at most 4,096 tokens per response; each request's seed is derived from the game and step and shared by all arms.

ALFWorld. The test set is the 134 `valid_unseen` games of six task types (24 Pick, 23 Heat, 21 Cool, 31 Clean, 18 Look and 17 PickTwo), as in prior ALFWorld agents (Yao et al., 2023); the development set is the 140 `valid_seen` games. Every arm uses the SkillRL prompt, history and action parser (App. C.1); success is the share of games won within 50 actions. TRIM's threshold for each backbone is set by the rule of App. C.3.

ToolQA. SRA-Bench ToolQA (Su et al., 2026; Zhuang et al., 2023) has questions that need external tools in seven domains; we use 50 questions from each of six of them, all but SciREX. The agent is the ReAct agent of SRA-Bench (at most 12 steps) with Qwen3's thinking decoding and at most 4,096 tokens per step. Skill is the frozen top-5 BM25 block of each question; prompting, SkillAligner and Planning handle it as on ALFWorld, and Planning's plan call also runs in thinking mode. TRIM is unchanged except for θ, which the rule of App. C.3 sets on 120 development questions disjoint from the 300. A question counts once its episode ends without an infrastructure error (a context overflow counts as a wrong answer), and infrastructure errors are re-run; Table 8 gives the correct answers per domain.

D.2 Baselines and Prompts

Every baseline receives the same five retrieved skills in the same prompt slot, so the arms differ only in how the block is handled.

Prompting. The prompting baseline puts one line directly above the BM25 block; the block itself is byte-identical to Skill.

```
You are checking the skill text given to an agent that acts in a text-based household environment
    (ALFWorld). The agent is in the middle of its episode.
## Task [G]
{GOAL}
## Harness facts
[H1] At every step the agent outputs one action inside <action></action>; only that action is executed.
    It should be one of the admissible actions listed for that step, otherwise the environment answers
    "Nothing happens." From step 2 on the agent sees the skill text below, its last 2 observation/action
    pairs and the current observation.
[H2] The episode succeeds as soon as the state described by the task [G] holds; nothing else is checked.
    The episode ends after 50 actions.
[H3] Objects and locations exist only if they appear in an observation or an admissible action, under
    those exact names (e.g. 'countertop 1', 'apple 2').
## Episode so far (actions and observations)
[O0] Initial observation: {...}   Action {k}: {...}   [O{k}] Observation {k}: {...}   (last 8 steps)
## Admissible actions at the current step [A]
{...}
## The agent's proposed next action (not executed yet) [P]
{DRAFTED ACTION}
## The agent's reasoning before [P] (end of it)
{LAST 1,500 CHARACTERS OF THE DRAFT'S REASONING}
## Skill text (line-numbered)
{S1:L1| ... }
## NEEDS (from the task sentence)
{NEEDS}
## Skill items (the state the agent currently sees)
I{k} = S1:L{a}-L{b} "{TITLE}": original | WITHHELD | REPLACED with: {...}
Added instruction in effect: none
## Your task
The skill text was retrieved for this task automatically and may not fit it. NEEDS below was written
    from the task sentence alone; use it as given.
Then give a verdict for EVERY skill item listed under 'Skill items', given what has been observed so far:
- KEEP: leave the item as the agent currently sees it (the default, also when unsure).
- WITHHOLD: following the item would make the agent do something NEEDS does not contain, or its condition
    can never hold here or is ruled out by the observations.
- REPLACE: the item's idea serves NEEDS but a detail is wrong for this task or episode; give the corrected
    instruction.
- RESTORE: an earlier change to this item was wrong; the agent should see the original item again.
If NEEDS requires a step or check that no item covers and the episode so far shows the agent missing it,
    give it as ADD (at most one).
- A condition in an item ('Apply when', 'after', 'once', 'before') may describe a LATER moment of the
    episode. Do not withhold an item only because its condition is not true yet; withhold it only if the
    condition can never hold for this task or the observations rule it out.
- Never withhold, and never leave withheld, an item that helps achieve something in NEEDS; if such an
    item is currently withheld, RESTORE it.
- Never RESTORE or ADD anything that asks for an action, object or change that NEEDS does not contain.
- Write instructions to the agent, not explanations. Use only actions, objects and locations that appear
    in the task, the observations, the admissible actions or the skill text.
- Every change must cite its evidence: [G] the task, [H1]/[H2]/[H3] a harness fact, [O<k>] observation k,
    [A] the admissible actions, [P] the proposed action.
Output exactly this and nothing else:
ITEMS:
- I<k> | KEEP, WITHHOLD, REPLACE or RESTORE | <evidence tag(s)> | <short reason, or the instruction for
    REPLACE; at most 25 words>
(one line per item, in order)
ADD: NONE, or ADD: <evidence tag(s)> | <instruction; at most 30 words>
```

Figure 6: The Editor's verdict prompt at the first firing on ALFWorld (template; blank lines removed). Only the opening line and the harness facts [H1]–[H3] describe ALFWorld; ToolQA replaces these lines and keeps every rule. NEEDS comes from an earlier call that sees only the task sentence: "NEEDS = everything the task sentence [G] asks for: the outcome, every object, place or value it names, and every property it attributes to an object. A property attributed to an object (for example 'a sorted list', 'a warm drink') is a state the agent must PRODUCE, not a state that already holds."

Prompting. These skills were written without seeing your environment: treat them as a reference, adapt them to the tools and feedback you actually get, and still do any step the task needs even if the skills omit it:

SkillAligner. We implement SkillAligner (Li et al., 2026a) from the prompts in its appendix. Before the episode, the agent's model makes one adaptation call (temperature 1 as in the paper, top-p 1, seed 0, at most 2,048 output tokens). Its input is the goal sentence, the five retrieved skills and, in the slot for the environment, ALFWorld's action format and step budget; it never sees the initial observation, the game identity or a solution. The output is a JSON object with the string lists *Primary*, *Checks*, *Avoid* and *Fallback*; a schema failure counts as a failed episode (3 of 133 Qwen3-8B episodes), without repair or retry. The guide replaces the raw block in the same slot from step 2 on and stays fixed.

Planning. Planning implements SkillX (Wang et al., 2026a): one planning call per episode, also in thinking mode, sees only the goal; each plan step is a BM25 query over the same index, and the hits fill the same slot up to five skills. It ran three times on ALFWorld with each backbone (Qwen3-8B: 56,

61 and 65; Qwen3-14B: 68, 75 and 75 of the 134 test games) and once on ToolQA with each.

Absolute success rates. SkillAligner reports, for Qwen3-8B on ALFWorld, 52.99% without skills, 61.94% with raw top-k skills and 80.60% with SkillAligner (Li et al., 2026a); in our harness the same backbone in thinking mode reaches 42.5%, 44.8% and 46.6%. The harnesses differ: we use SkillRL's released ALFWorld agent loop (its prompt, two steps of history, at most 50 actions), whereas SkillAligner uses its own prompts and execution protocol. All methods in our tables share one harness.

Methods not compared. SelSkill (Chen et al., 2026) releases training and evaluation code but no trained selector checkpoint, and reproducing it requires its GRPO initialization and three rounds of preference training; SkillsInjector (Li et al., 2026c) likewise trains the planner that decides which skills to inject. Both fall outside a training-free comparison.

D.3 Serving and Computational Cost

TRIM generates no more tokens than Skill (Table 10); its overhead is two prefills per scored step, 79.6k prompt tokens per episode and 8.5% of the episodes' wall-clock time, and 1.4k Editor tokens per episode. Because the skill-on prompt is the generation prompt, its log-probabilities can be read during generation, which removes one of the two prefills.

Servers. NoSkill, Skill, prompting and SkillAligner ran on ALFWorld on a vLLM (Kwon et al., 2023) 0.8.5 server with two RTX A5000 GPUs (tensor parallel); TRIM, PRE, RAND, every development arm, the restored-state tests and all ToolQA arms ran on A100 80GB GPUs with vLLM 0.29.0. Both servers apply Qwen3's chat template with thinking enabled and separate the reasoning with vLLM's reasoning parser (`deepseek_r1` and `qwen3`), and both draw the first draft of each step with the same seed formula, so arms share seeds at the same game and step. The two servers give Skill the same development success within one game (71 and 70 of 140 games).

Runs. TRIM, PRE and RAND ran three times on the test games (seed bases 1701, 1702 and 1703); NoSkill, Skill, prompting and SkillAligner with Qwen3-8B ran once with TRIM's first seed base, and each Qwen3-14B baseline three times with one configuration and server (Table 7); the Qwen3-14B Planning runs on ALFWorld use the same server. TRIM with Qwen3-14B uses seed bases 1701–1703 on the test games, with $\theta = 1.1423$ set by the rule of §4.1 on an observation run of Qwen3-14B on the development games. Episodes that ended in an infrastructure error were re-run; one SkillAligner game (Qwen3-8B) failed in all three attempts and is excluded from that arm's counts and paired tests.

D.4 Statistical Tests

Every comparison is paired per game, because reruns with fixed seeds change 11% of outcomes (App. B.3). For two arms, X:Y counts the games that only one of them wins, or, with three runs of each, the games one arm wins more often; p is a two-sided exact sign test on these games (Dror et al., 2018). Effects Δ are differences in per-game success, in percentage points, where a game's success is its win rate over the arm's runs, with 95% percentile bootstrap intervals over games (Koehn, 2004) (10,000 resamples; 4,000 over episodes or states for the restored-state tests). We correct each family of paired tests with Holm's method (Holm, 1979): the seven run-1 comparisons of Qwen3-8B and the five comparisons of Qwen3-14B (Table 9), and the six development comparisons of TRIM with the arms of Table 4 and with the two stages at random steps. Qwen3-14B arms are compared three runs against three runs, game by game.

E Additional Results

This section reports every run, task type and paired test behind §5.2 and §5.3.

E.1 Results per Run, Task Type and Backbone

TRIM wins the most games in every run with both backbones: with Qwen3-8B it wins 84, 80 and 75 of the 134 test games, on average 19.7 more than Skill's 60, and with Qwen3-14B 84, 91 and 86 (64.9%; 62.7% in run 1), against at most 75 in any run of a baseline (Table 7). Its largest gains over the retrieved skills in run 1 are on Cool (81.0% against 52.4%) and Pick (66.7% against 37.5%). With Qwen3-8B, its first run is ahead of every baseline and of both controls (between 24:6 games against prompting and 30:2 against Planning; $p \leq 0.003$, also after Holm), and with Qwen3-14B, comparing three runs of each arm game by game, it is ahead of NoSkill (58:11), Skill (47:6), prompting

Arm	Run	Pick	Heat	Cool	Clean	Look	Two	All
NoSkill	r1	16/24	9/23	11/21	15/31	10/18	0/17	61/134
	r2	18/24	8/23	10/21	12/31	8/18	1/17	57/134
	r3	18/24	10/23	7/21	11/31	9/18	2/17	57/134
Skill	r1	14/24	6/23	10/21	20/31	12/18	1/17	63/134
	r2	17/24	7/23	12/21	20/31	12/18	1/17	69/134
	r3	18/24	6/23	9/21	20/31	13/18	1/17	67/134
Prompting	r1	16/24	7/23	12/21	18/31	14/18	1/17	68/134
	r2	17/24	5/23	9/21	18/31	12/18	1/17	62/134
	r3	16/24	9/23	11/21	20/31	12/18	0/17	68/134
SkillAligner	r1	16/24	4/23	9/21	21/31	9/18	3/17	62/134
	r2	20/24	7/23	9/21	19/31	12/18	4/17	71/134
	r3	19/24	6/23	12/21	20/31	12/18	3/17	72/134
Planning	r1	16/24	7/23	13/21	19/31	13/18	0/17	68/134
	r2	15/24	12/23	13/21	21/31	13/18	1/17	75/134
	r3	16/24	8/23	15/21	21/31	14/18	1/17	75/134
TRIM	r1	20/24	8/23	15/21	23/31	14/18	4/17	84/134
	r2	20/24	10/23	17/21	24/31	15/18	5/17	91/134
	r3	20/24	10/23	16/21	22/31	14/18	4/17	86/134

Table 7: Qwen3-14B on the ALFWorld test games: wins / games per run and task type (thinking mode; Two: PickTwo).

Model	Arm	Flt.	Cof.	Yelp	Abnb.	Agd.	DBLP	All
Qwen3-8B	NoSkill	7/50	19/50	24/50	11/50	8/50	10/50	79/300
	Skill	27/50	29/50	25/50	20/50	4/50	10/50	115/300
	Prompting	26/50	28/50	25/50	20/50	3/50	13/50	115/300
	SkillAligner	25/50	20/50	26/50	16/50	4/50	5/50	96/300
	Planning	20/50	25/50	21/50	18/50	4/50	10/50	98/300
	TRIM	30/50	34/50	36/50	22/50	4/50	14/50	140/300
Qwen3-14B	NoSkill	18/50	22/50	28/50	18/50	5/50	15/50	106/300
	Skill	25/50	36/50	20/50	16/50	3/50	16/50	116/300
	Prompting	26/50	35/50	20/50	14/50	4/50	10/50	109/300
	SkillAligner	27/50	26/50	26/50	21/50	0/50	7/50	107/300
	Planning	23/50	27/50	20/50	10/50	5/50	11/50	96/300
	TRIM	31/50	37/50	34/50	20/50	4/50	15/50	141/300

Table 8: ToolQA in thinking mode: correct answers / questions per domain (300 questions, 50 per domain; one run per arm).

(46:5), SkillAligner (49:11) and Planning (42:19; $p < 0.005$, also after Holm; Table 9). On ToolQA, TRIM answers 140 and 141 of the 300 questions with Qwen3-8B and Qwen3-14B (Table 8), more than every baseline with both backbones (against Skill, 31:6 and 34:9 questions, both $p < 0.001$).

Figure 7 gives the per-game margin of TRIM's three runs over each baseline, and Table 10 the lost wins of Figure 3 with the length and generation of each arm.

E.2 Timing Controls

PRE and RAND share TRIM's runner, Editor, regeneration and withdrawal rule and differ from it only in when they intervene: PRE fires once, at step 2, so its second stage never occurs, and RAND applies both stages at random steps. TRIM beats both over three runs (41:11 and 34:11 games, both $p < 0.001$), so its gain comes from when it intervenes, not from the edit and the withdrawal themselves. RAND draws a seeded coin at every step

TRIM	X:Y	p	Holm	Δ	95% CI
Qwen3-8B, run 1					
vs. PRE	28:7	<0.001	0.002	+15.7	[7.5, 23.9]
vs. RAND	25:7	0.002	0.003	+13.4	[5.2, 21.6]
vs. Skill	28:4	<0.001	<0.001	+17.9	[10.4, 25.4]
vs. NoSkill	30:3	<0.001	<0.001	+20.1	[12.7, 27.6]
vs. prompting	24:6	0.001	0.003	+13.4	[6.0, 20.9]
vs. SkillAligner	32:10	<0.001	0.003	+16.5	[7.5, 25.6]
vs. Planning	30:2	<0.001	<0.001	+20.9	[13.4, 28.4]
Skill vs. NoSkill	24:21	0.766	–	+2.2	[−7.5, 11.9]
RAND vs. PRE	13:10	0.678	–	+2.2	[−4.5, 9.0]
Qwen3-8B, runs 1–3					
vs. PRE	41:11	<0.001	–	+11.7	[6.5, 16.9]
vs. RAND	34:11	<0.001	–	+8.7	[4.2, 13.4]
Qwen3-14B, runs 1–3					
vs. NoSkill	58:11	<0.001	<0.001	+21.4	[14.7, 28.1]
vs. Skill	47:6	<0.001	<0.001	+15.4	[10.7, 20.4]
vs. prompting	46:5	<0.001	<0.001	+15.7	[11.2, 20.4]
vs. SkillAligner	49:11	<0.001	<0.001	+13.9	[8.2, 19.7]
vs. Planning	42:19	0.004	0.004	+10.7	[4.7, 16.7]

Table 9: Paired tests on the 134 test games (133 for SkillAligner with Qwen3-8B). X:Y: games won by one arm only; over runs 1–3, games one arm wins more often. p: exact sign test; Holm: over the seven Qwen3-8B run-1 tests and over the five Qwen3-14B tests; Δ: success difference (points) with a bootstrap CI over games. Qwen3-8B runs 1–3 against the baselines: Figure 7. Skill against NoSkill and RAND against PRE are shown for reference and are not in a Holm family. TRIM beats every arm it is compared with.

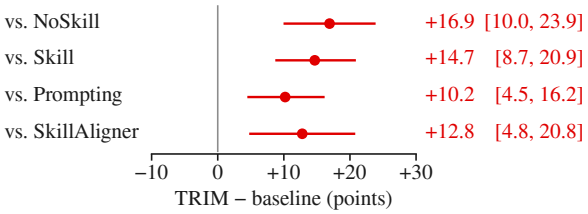


Figure 7: TRIM against each baseline on the 134 test games (Qwen3-8B, thinking): per-game success of TRIM averaged over its three runs (84, 80 and 75 wins) minus the baseline's single run, in points, with 95% bootstrap CIs over games (133 games for SkillAligner); run-1 tests: App. Table 9.

from step 2 on with $p = 0.0289$, TRIM's number of firings per scored step on the development games, so both arms fire at a matched rate (App. F.1); the coin depends only on the game and step. Over the three runs TRIM wins 239 of 402 test episodes, against 192 for PRE and 204 for RAND (Table 11).

E.3 Development Runs and Ablations

All development arms ran on the 140 `valid_seen` games with the seeds of the observation run; one TRIM episode did not finish, so arms are compared

Arm	Lost of NoSkill	Lost of Skill	Steps	Tokens Gen. (k)
NoSkill	–	24	11.5	38.1
Skill	21	–	8.2	32.0
+ prompting	18	7	9.4	30.5
SkillAligner	20	18	9.3	35.8
TRIM	3	4	9.3	31.0

Table 10: Lost wins, length and generation (Qwen3-8B, thinking, test run 1). Lost: wins of NoSkill (57) or Skill (60) that the arm loses. Steps: mean actions on the 26 games all arms solve. Gen.: generated tokens per episode, with Editor calls.

Arm	Pick	Heat	Cool	Clean	Look	Two	All	Runs
PRE	62.5	39.1	57.1	52.7	50.0	15.7	47.8	63, 64, 65
RAND	51.4	50.7	60.3	59.1	57.4	15.7	50.7	66, 69, 69
TRIM	66.7	58.0	74.6	63.4	55.6	29.4	59.5	84, 80, 75

Table 11: Success (%) over three test runs (402 episodes) of a one-time edit at injection (PRE), of the two stages at random steps (RAND) and of TRIM, per task type (Qwen3-8B, thinking; Two: PickTwo). Runs: wins in runs 1, 2 and 3. PRE reviews at step 2; RAND fires with $p = 0.029$ per step, TRIM's development rate, so the two arms fire at a matched rate. TRIM wins the most episodes in every run.

Arm	Won	Harmed	Needed	Fir.
No edit (Skill)	70/140	0/15	20/20	0.00
Edits only (no withdrawal), at the steps of				
S_{lr}	66/140	3/15	15/20	1.46
JSD	62/140	4/15	12/20	1.09
Entropy	65/140	4/15	12/20	1.09
Combination	66/140	4/15	15/20	1.16
Two-stage gate	65/140	3/15	16/20	1.33
Random steps	70/140	5/15	17/20	1.59
Step 2	63/140	3/15	11/20	1.00
Candidate interventions at the S_{lr} firings				
Reasoning, $\leq$3 reviews	68/139	3/15	18/20	1.03
Edit, then withdraw	**86/139**	10/15	18/19	0.91
Withdraw at first firing	71/140	6/15	10/20	0.61
Attribution, one item	70/140	3/15	17/20	1.17
Arbitration	71/139	3/15	18/20	1.06
Two stages, random steps	73/140	5/15	17/20	1.10
Ablations of TRIM				
Edit only (one review)	69/140	3/15	14/20	0.61
Regenerate, then withdraw	68/140	5/15	15/20	0.92
Edit, random withdrawal	72/140	4/15	15/20	0.91

Table 12: Development arms (140 `valid_seen` games, Qwen3-8B, thinking). Harmed: games Skill lost in both of its runs and NoSkill won at least once; Needed: Skill won both, NoSkill at most one. Fir.: firings per episode. Bold: TRIM, chosen by a rule registered before the candidates' runs; the ablations were registered before their runs. TRIM wins the most games and rescues the most harmed games while keeping the games that need the skills.

with TRIM on the 139 games both finished (Table 12). The strata come from two runs each of Skill and NoSkill: the 15 games the skills harm (App. B.2) and the 20 that need them.

With edits alone, each made by an Editor that does not read the reasoning excerpt, no trigger beats the unedited block (62–70 against 70 of 140 development games) and none rescues more than 5 of the 15 harmed games; the withdrawal at the second firing rescues 10, which is why TRIM has two stages. Five candidate interventions at the S_{lr} firings were compared by a rule registered before their results (highest development success, ties broken by fewer firings per episode); edit-then-withdraw won (86 of 139; 0.614 even if its unfinished episode were lost, against at most 0.511). It also beats the unedited block (18:1 games) and the same two stages at random steps, run as its control on these games (13:0).

After the choice, three component ablations were registered and run once each with the same seeds: one review without withdrawal, a plain regeneration of the step instead of the edit at the first firing, and the edit followed by a withdrawal at a random step ($p = 0.0318$ per step, TRIM's rate of second firings on the development games). Each ablation lowers development success: without the withdrawal (17:0 games), regenerating instead of editing (21:3) and withdrawing at a random step after the edit (17:3). Withdrawing at the first firing loses to TRIM (18:3 games) and keeps 10 of the 20 games that need the skills, against 18 of 19.

E.4 Pre-registered Criterion and Restored-State Tests

Before any repeat run, we registered a success criterion: a mean margin of at least five wins over Skill's 60 and a win over PRE in every run. TRIM meets both conditions: its mean of 79.7 wins exceeds Skill's by 19.7, and it beats PRE in every run (84, 80 and 75 wins against 63, 64 and 65). It also beats RAND over the three runs (34:11 games, $p < 0.001$).

Before their results we also registered tests on the development games (Table 13). Each restores logged episodes by replaying their actions, so every restored prompt equals the logged one, and continues them to the 50-step limit twice per branch with fixed seeds and no further intervention. *Timing*: in

Comparison (development)	n	Δ	95% CI
Timing: d = win rate withdrawn − kept			
d(signal) − d(nearest)	74	+12.8	[4.1, 20.9]
d(signal) − d(random)	74	+16.2	[6.8, 25.7]
*d by firings up to step 40**			
≥2 firings	41	+19.9	[7.3, 32.5]
one firing	33	−11.6	[−20.7, −3.0]
difference	41/33	+31.5	[16.2, 46.8]
d(signal) by stratum (episodes)			
harmed +57.7 (13), needed −34.6 (13), easy −7.1 (14), mixed +18.8 (8), fail either way +28.0 (25)			
Second firing: win rate of continuations			
withdraw − keep edited	42	+11.9	[1.2, 22.6]
restore original (33): +9.1 [−3.0, 21.2] against keep, −4.5 [−18.2, 9.1] against withdraw			

Table 13: Restored-state tests on development states (Qwen3-8B, thinking; two continuations per branch). Δ in points; bootstrap CIs over episodes or states. Withdrawal helps most at the signal's state and where the signal fires repeatedly. *Grouping defined on these runs; it holds on the test games (App. E.4). Strata cover 73 of the 74 episodes.

each of the 74 observation-run episodes in which the signal fired by step 40, three states are restored: the signal's (its second firing, or its only one), the nearest step without a firing and a random step without a firing. Each continues with the block kept and with it withdrawn, and d is the win rate without the block minus that with it. Withdrawing at the signal's state helps more than at the nearest or a random state (+12.8 and +16.2 points). *Second firing*: at the 42 states where TRIM withdrew the block, the continuations keep the edited block, withdraw it or restore the original block (at the 33 states whose block the edit had changed). Withdrawing beats keeping the edited block by +11.9 points ([1.2, 22.6]); restoring the original block lies between the two, 9.1 points above keeping and 4.5 below withdrawing.

Grouped by whether the signal fired at least twice, withdrawal helps in the 41 episodes with repeated firing (+19.9, [7.3, 32.5]) and hurts in the 33 with one firing (−11.6, [−20.7, −3.0]; difference +31.5, [16.2, 46.8]; §6). The grouping was defined on these development runs. A check registered before its observation run tests it on the test games, restoring every episode not finished by step 10 and grouping it by its firings up to then: on the 12 test states whose signal had fired at least twice, withdrawal raises the win rate by 35.4 points more than on the 97 others (95% CI [5.3, 65.0]).

	PRE	RAND	TRIM
Episodes with a firing	134	78	81
edits changing the block	125	77	64
withdrawn episodes	0	37	34
First firing: median [IQR]	2	14 [6, 28]	4 [2, 10]
Withdrawal: median [IQR]	–	27 [17, 41]	15 [9, 20]
Wins: no firing	–	47/56	29/53
firing, not withdrawn	63/134	11/41	33/47
withdrawn	–	8/37	22/34

Table 14: Intervention timing at a matched firing rate (Qwen3-8B, thinking, test run 1, 134 games). TRIM and RAND fire 0.86 times per episode; TRIM fires ten steps and withdraws twelve steps earlier (medians). IQR: interquartile range of the step.

F Additional Analysis

This section analyzes when the arms intervene and what each stage of TRIM removes.

F.1 Intervention Timing

At a matched firing rate, TRIM fires ten steps and withdraws twelve steps earlier than RAND: in run 1 both fire 0.86 times per episode, TRIM first at a median step of 4 and RAND at 14, and they withdraw at 15 and 27 (Table 14; the firing rate of RAND is defined in App. E.2). The ordering holds in every run: TRIM's first firing has a median step of 4 in all three runs and its withdrawal of 15, 11.5 and 11.5, against 14, 12 and 15 and 27, 24 and 29 for RAND. Over the three runs TRIM fires 350 times (0.87 per episode: 246 first firings and 104 withdrawals) and RAND 316 times (0.79: 213 and 103). Restored development states hold the episode fixed and isolate the effect of this timing: withdrawing at the signal's state beats withdrawing at the nearest state or a random state without a firing by +12.8 and +16.2 points (App. E.4). PRE changes 125 of the 134 blocks at step 2 and wins 63 games: an edit at injection does not undo the misfit. Episodes without a firing are defined differently in the two arms: RAND's end before its coin comes up, and 47 of its 56 are wins; TRIM's are those in which the block never moves the drafted response past θ.

F.2 Effect of Each Stage

The two stages remove different failures: the edit removes procedures written for other task types, and the withdrawal removes the search routines that the edit keeps. Over 1,840 first reviews in development runs of the edit-only variants, an Editor that sees the task sentence and the episode but not

the reasoning excerpt keeps 998 items, withholds 732 and replaces 110. It withholds most items written for other task types (410 of 574) but almost no generic search items (46 of 715); it keeps "Open Before Judging Empty" in all 134 verdicts on it. Without the reasoning, it also withholds half of the items for the task's own operation (276 of 551), 174 of them with a reason that denies the operation is needed, such as "already clean", and 89% of the evidence it cites for its first edits is the task sentence alone. Edits act on the plan before the action: over the edit-only development arms, in 109 of the 129 drafts that planned an operation of another task type, the regenerated reasoning no longer does, while the action of the step changes in 43 of 477 accepted edits. The skills' influence shows in the reasoning before it reaches the action tokens, which is why the Trigger scores the whole response (§4.1). On the test games, TRIM's Editor, which reads the reasoning excerpt, changes the block in 64 of its 81 first reviews: it withholds or replaces 57 of the 79 items written for another task type and withholds fewer items for the task's own operation (26 of 86, against half without the reasoning). The Editor removes few search items (7 of 49); the withdrawal at the second firing removes the whole block, search items included. In the games of Figures 2 and 8, the agent leaves the search routine within seven steps of the withdrawal.

G Case Studies

The cases come from test run 1 with Qwen3-8B in thinking mode and show both stages in one game, an edit that removes another task's procedure, and a withdrawal after a review that keeps the block.

Both stages in one game (Figure 2). Task: "cool some potato and put it in microwave." The block holds Open Before Judging Empty, Track Counts & Progress, Open Then Heat (a heating procedure), Systematic Object Hunt and Secure Exact Target First. With TRIM, S_{lr} stays between 0.31 and 0.84 at steps 2–9 while the agent searches the cabinets, and most drafts cite a skill item. At step 10 ("open cabinet 5", 1.12), the Editor withholds the heating procedure Open Then Heat and keeps the four other items. S_{lr} falls to 0.39 at step 11 and reaches 1.66 at step 12 ("open cabinet 6"), whose draft reads "according to the systematic object hunt skill, I should sweep every unvisited surface and open containers until the exact object is found"; the block is withdrawn. The agent revisits the fridge and three

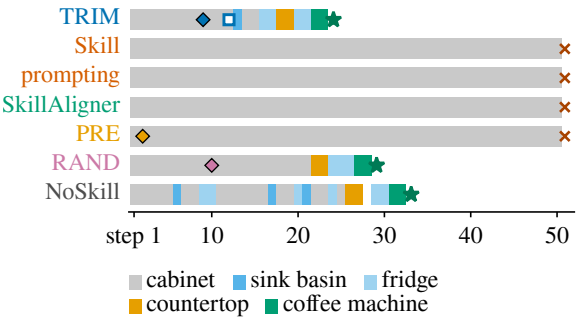


Figure 8: Where each arm goes in one test game, "cool some mug and put it in coffeemachine". Diamond: edit; square: withdrawal; star: goal done; cross: 50 steps used. PRE: TRIM's Editor once, at step 2; RAND: both stages at random steps. TRIM's first 12 prompts equal Skill's; after the withdrawal at step 12, it leaves the cabinets at the next step and wins.

cabinets, goes to the sink basin at step 18 and to the countertop at step 19 ("Maybe the potato is on the countertop."), and wins at step 25. Skill shares the first six actions and then visits only the fridge and cabinets 1–6 until step 50; its reasoning cites skill items in 46 of 49 steps. NoSkill goes to the countertop at step 3 and wins at step 9; PRE makes the same edit at step 2 and loses; SkillAligner's guide turns the heating item into "Cool the potato with the microwave" and loses; prompting loses.

The edit removes another task's procedure. Task: "put a peppershaker in drawer." The block holds Open Before Judging Empty, Track Counts & Progress, Lock In Observed Items, Single Toggle Rule (a desk-lamp procedure) and Enforce Cooling Before Placement, which forbids placing the target before cooling it with the fridge. With TRIM, the signal fires at step 3 ($S_{\text{lr}} = 1.66$), and the Editor withholds the cooling item ("No fridge cooling is required for placing a peppershaker in a drawer."), the lamp item and Lock In Observed Items, and keeps the two general principles. The cooling procedure never reaches the agent's prompt after step 3; at step 15 the signal fires again and the block is withdrawn. The agent finds the peppershaker on the countertop at step 23, picks it up at step 24 and places it in the drawer at step 27. Skill shares the first six actions and picks up the same peppershaker at step 20, but then goes to the fridge, opens three drawers without placing it, and reasons at step 29 that "the 'Enforce Cooling Before Placement' principle says not to place the target until after cooling with the fridge"; it never places the peppershaker and loses at step 50. NoSkill also loses.

The same history until the withdrawal (Figure 8). Task: "cool some mug and put it in coffeemachine." For steps 1–12, TRIM's prompts are identical to Skill's. At step 9 ($S_{\mathrm{lr}} = 1.67$) the review keeps the block; at step 12 ($S_{\mathrm{lr}} = 1.16$) the signal fires again and the block is withdrawn. At step 13 the agent leaves the cabinets for the sink basin, finds the mug at step 18 and wins at step 23, whereas Skill, SkillAligner, prompting and PRE visit only cabinets 1–6 for 50 steps. NoSkill wins at step 32.